Also note from Eq. (5.51) that the determinant of $\mathbf{P}'$ is unity, and that $(\mathbf{P}')^{-1}$ is obtained merely by replacing all the elements above the main diagonal of $\mathbf{P}'$ by their *negatives*. Substituting Eq. (5.50) into Eq. (5.47), the regulator gain matrix is thus given by

$$\mathbf{K} = (\boldsymbol{\alpha} - \mathbf{a})\mathbf{P}'\mathbf{P}^{-1} \tag{5.52}$$

Equation (5.52) is called the *Ackermann's pole-placement formula*. For a single-input plant considered here, both $\mathbf{P}$ and $\mathbf{P}'$ are square matrices of size $(n \times n)$. Note that if the plant is uncontrollable, $\mathbf{P}$ is singular, thus $\mathbf{T} = \mathbf{P}'\mathbf{P}^{-1}$ *does not exist*. This confirms our earlier requirement that for pole-placement, a plant must be controllable.

Example 5.9

Let us design a full-state feedback regulator for an inverted pendulum on a moving cart (Figure 2.59). A linear state-space representation of the plant is given by Eqs. (3.31) and (3.32), of which the state coefficient matrices are the following:

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ (M+m)g/(ML) & 0 & 0 & 0 \\ -mg/M & 0 & 0 & 0 \end{bmatrix}; \quad \mathbf{B} = \begin{bmatrix} 0 \\ 0 \\ -1/(ML) \\ 1/M \end{bmatrix}$$

$$\mathbf{C} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}; \quad \mathbf{D} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{5.53}$$

The single-input, $u(t)$, is a force applied horizontally to the cart, and the two outputs are the angular position of the pendulum, $\theta(t)$, and the horizontal position of the cart, $x(t)$. The state-vector of this fourth order plant is $\mathbf{x}(t) = [\theta(t);\ x(t);\ \theta^{(1)}(t);\ x^{(1)}(t)]^T$. Let us assume the numerical values of the plant's parameters as follows: $M = 1$ kg, $m = 0.1$ kg, $L = 1$ m, and $g = 9.8$ m/s^2. Then the matrices $\mathbf{A}$ and $\mathbf{B}$ are the following:

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 10.78 & 0 & 0 & 0 \\ -0.98 & 0 & 0 & 0 \end{bmatrix}; \quad \mathbf{B} = \begin{bmatrix} 0 \\ 0 \\ -1 \\ 1 \end{bmatrix} \tag{5.54}$$

Let us first determine whether the plant is controllable. This is done by finding the controllability test matrix, $\mathbf{P}$, using the MATLAB (CST) command *ctrb* as follows:

```
>>P = ctrb(A,B)) <enter>

P =

0          -1.0000    0          -10.7800
0          1.0000     0          0.9800
-1.0000    0          -10.7800   0
1.0000     0          0.9800     0
```

The determinant of the controllability test matrix is then computed as follows:

```
>>det(P) <enter>

ans =
-96.0400
```

Since $|\mathbf{P}| \neq 0$, it implies that the plant is controllable. However, the *magnitude* of $|\mathbf{P}|$ depends upon the *scaling* of matrix **P**, and is *not* a good indicator of how *far away* **P** is from being singular, and thus how *strongly* the plant is controllable. A better way of detecting the measure of controllability is the condition number, obtained using the MATLAB function *cond* as follows:

```
>>cond(p) <enter>

ans =
 12.0773
```

Since condition number of **P** is *small* in magnitude, the plant is *strongly* controllable. Thus, our pole-placement results are expected to be accurate. (Had the condition number of **P** been *large* in magnitude, it would have indicated a *weakly* controllable plant, and the inversion of **P** to get the feedback gain matrix would have been inaccurate.) The poles of the plant are calculated by finding the eigenvalues of the matrix **A** using the MATLAB command *damp* as follows:

```
>>damp(A) <enter>

Eigenvalue      Damping      Freq. (rad/sec)
 3.2833         -1.0000      3.2833
    0           -1.0000      0
    0           -1.0000      0
-3.2833          1.0000      3.2833
```

The plant is *unstable* due to a pole with positive real-part (and also due to a pair of poles at $s = 0$). Controlling this unstable plant is like balancing a vertical stick on your palm. The task of the regulator is to stabilize the plant. Let us make the closed-loop system stable, by selecting the closed-loop poles as $s = -1 \pm i$, and $s = -5 \pm 5i$. The coefficients of the plant's characteristic polynomial can be calculated using the MATLAB command *poly* as follows:

```
>>a=poly(A) <enter>

a =

      1.0000   0.0000   -10.7800   0   0
```

which implies that the characteristic polynomial of the plant is $s^4 - 10.78s^2 = 0$. Hence, the polynomial coefficient vector, **a**, is the following:

$$\mathbf{a} = [0; \quad -10.78; \quad 0; \quad 0] \tag{5.55}$$

The characteristic polynomial of the closed-loop system can also be calculated using the command *poly* as follows:

```
>>v = [-1+j; -1-j; -5+5*j; -5-5*j]; alpha = poly(v) <enter>

alpha =

  1  12  72  120  100
```

which implies that the closed-loop characteristic polynomial is $\alpha^4 + 12\alpha^3 + 72\alpha^2 + 120\alpha + 100$, and the vector $\boldsymbol{\alpha}$ is thus the following:

$$\boldsymbol{\alpha} = [12; \quad 72; \quad 120; \quad 100] \tag{5.56}$$

Note that the MATLAB function *poly* can be used to compute the characteristic polynomial either *directly* from a square matrix, or from the *roots* of the characteristic polynomial (i.e. the eigenvalues of a square matrix). It now remains to find the upper triangular matrix, $\mathbf{P}'$, by either Eq. (5.49) or Eq. (5.51). Since a controller companion form is generally ill-conditioned (see Chapter 3), we would like to avoid using Eq. (5.49) which involves higher powers of the ill-conditioned matrix, $\mathbf{A}'$. From Eq. (5.51), we get

$$\mathbf{P}' = \begin{bmatrix} 1 & 0 & 10.78 & 0 \\ 0 & 1 & 0 & 10.78 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{5.57}$$

Finally, the regulator gain matrix is obtained through Eq. (5.52) as follows:

```
>>Pdash=[1 0 10.78 0; 0 1 0 10.78; 0 0 1 0; 0 0 0 1]; a=[0 -10.78 0 0];

              alpha=[12 72 120 100]; K = (alpha-a)*Pdash*inv(P) <enter>

K =

-92.9841   -10.2041   -24.2449   -12.2449
```

The regulator gain matrix is thus the following:

$$\mathbf{K} = [-92.9841; \quad -10.2041; \quad -24.2449; \quad -12.2449] \tag{5.58}$$

Let us confirm that the eigenvalues of the closed-loop state-dynamics matrix, $\mathbf{A}_{CL} = \mathbf{A} - \mathbf{BK}$, are indeed what we set out to achieve as follows:

```
>>ACL = A-B*K <enter>

ACL =
 0              0              1.0000         0
 0              0              0              1.0000
 -82.2041       -10.2041       -24.2449       -12.2449
 92.0041        10.2041        24.2449        12.2449
```

The closed-loop poles are then evaluated by the command *eig* as follows:

```
>>eig(ACL) <enter>

ans =
 -5.0000+5.0000i
 -5.0000-5.0000i
 -1.0000+1.0000i
 -1.0000-1.0000i
```

Hence, the desired locations of the closed-loop poles have been obtained.

The computational steps of Example 5.9 are programmed in the MATLAB (CST) function called *acker* for computing the regulator gain matrix for single-input plants using the Ackermann's formula (Eq. (5.52). The command *acker* is used as follows:

```
>>K = acker(A,B,V) <enter>
```

where A, B are the state coefficient matrices of the plant, V is a vector containing the desired closed-loop pole locations, and K is the returned regulator gain matrix. Since Ackermann's formula is based on transforming the plant into the controller companion form, which becomes ill-conditioned for large order plants, the computed regulator gain matrix may be inaccurate when n is greater than, say, 10. The command *acker* produces a warning, if the computed closed-loop poles are *more than* 10% *off* from their desired locations. A similar MATLAB (CST) function called *place* is also available for computing the pole-placement regulator gain for single-input plants. The function *place* also provides an output *ndigits*, which indicates the number of significant digits to which the closed-loop poles have been placed. The design of Example 5.9 is simply carried out by using the command *place* as follows:

```
>>V = [-1+j; -1-j; -5+5*j; -5-5*j]; K = place(A,B,V) <enter>

place: ndigits= 17

K =

-92.9841   -10.2041   -24.2449   -12.2449
```

The result is identical to that obtained in Example 5.9; $ndigits = 17$ indicates that the locations of the closed-loop poles match the desired values up to 17 significant digits.

The locations of closed-loop poles determine the performance of the regulator, such as the settling time, maximum overshoot, etc. (see Chapter 2 for performance parameters) when the system is disturbed by a non-zero initial condition. A design is usually specified in terms of such performance parameters, rather than the locations of the closed-loop poles themselves. It is the task of the designer to ensure that the desired performance is achieved by selecting an appropriate set of closed-loop poles. This is illustrated in the following example.

Example 5.10

For the inverted-pendulum on a moving cart of Example 5.9, let us design a regulator which achieves a 5% maximum overshoot and a settling time less than 1 second for both the outputs, when the cart is initially displaced by 0.01 m. The state coefficient matrices, **A**, **B**, **C**, and **D**, of the plant are given in Eq. (5.53). The initial condition vector has the perturbation to the cart displacement, $x(t)$, as the only non-zero element; thus, $\mathbf{x}(0) = [0; 0.01; 0; 0]^T$. Let us begin by testing whether the regulator designed in Example 5.9 meets the performance specifications. This is done by using the MATLAB (CST) function *initial* to find the initial response as follows:

```
>>t = 0:0.1:10; sysCL=ss(A-B*K, zeros(4,1),C,D); [y,t,X] = initial
  (sysCL,[0 0.01 0 0]',t); <enter>
```

where **y**, **X**, and **t** denote the returned output, state, and time vectors and *sysCL* is the state-space LTI model of the closed-loop system. The resulting outputs $\mathbf{y}(t) = [\theta(t); x(t)]^T$ are plotted in Figure 5.4.

In Figure 5.4, both the responses are seen to have acceptably small maximum overshoots, but settling-times in excess of 5 s, which is unacceptable. In order to *speed-up* the closed-loop response, let us move all the poles *deeper* inside the left-half plane by decreasing their real parts such that the new desired closed-loop poles are $s = -7.5 \pm 7.5i$, and $s = -10 \pm 10i$. Then, the new regulator gain matrix, the closed-loop dynamics matrix, and the initial response are obtained as follows:

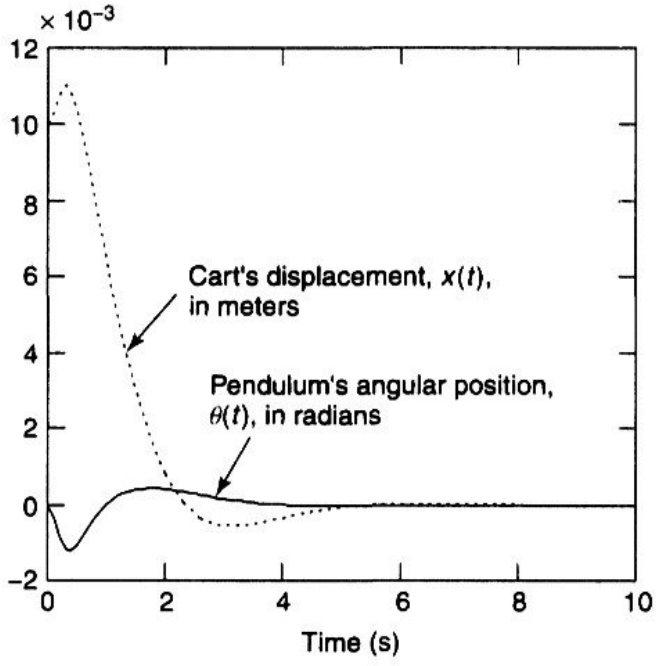


Figure 5.4 Closed-loop initial response of the regulated inverted pendulum on a moving cart to perturbation on cart displacement for the regulator gain matrix, **K** = [−92.9841; −10.2041; −24.2449; −12.2449]

```
>>V=[-7.5+7.5*j -7.5-7.5*j -10+10*j -10-10*j]'; K = place(A,B,V) <enter>

place: ndigits= 19

K =

-2.9192e+003   -2.2959e+003   -5.7071e+002   -5.3571e+002
>>t = 0:0.01:2; sysCL=ss(A-B*K, zeros(4,1),C,D); [y,t,X] = initial(sysCL,
  [0 0.01 0 0]',t); <enter>
```

The resulting outputs are plotted in Figure 5.5, which indicates a maximum overshoot of the steady-state values less than 4%, and a settling time of less than 1 s for both the responses.

How did we know that the new pole locations will meet our performance requirements? We didn't. We tried for several pole configurations, until we hit upon the one that met our requirements. This is the design approach in a nutshell. On comparing Figures 5.4 and 5.5, we find that by moving the closed-loop poles further inside the left-half plane, we *speeded-up* the initial response *at the cost* of increased maximum overshoot. The settling time and maximum overshoot are, thus, *conflicting requirements*. To decrease one, we have to accept an increase in the other. Such a compromise, called a *trade-off*, is a hallmark of control system design. Furthermore, there is another cost associated with moving the poles deeper inside the left-half plane – that of the control input. Note that the new regulator gain elements are *several times larger* than those calculated in Example 5.9, which implies that the regulator must now apply an input which is much

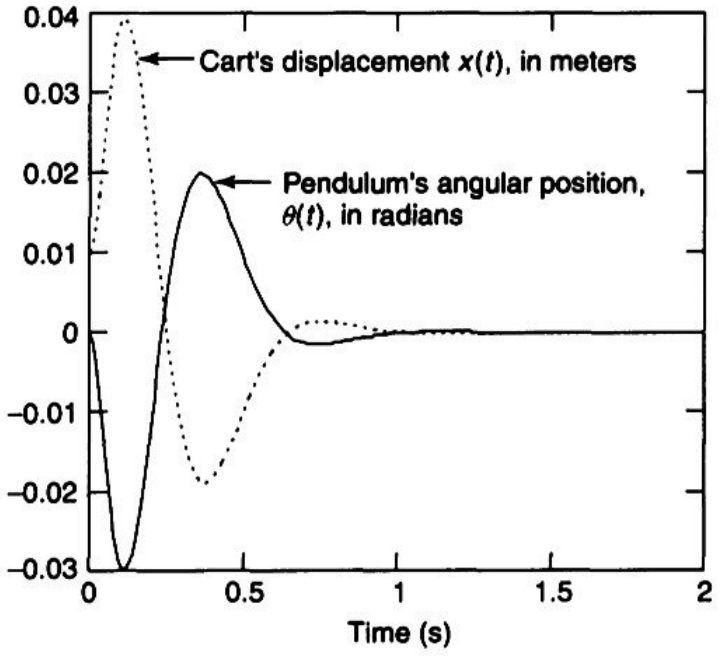


Figure 5.5 Closed-loop initial response of the regulated inverted pendulum on a moving cart to perturbation on cart displacement for the regulator gain matrix, **K** = [−2919.2; −2295.9; −570.71; −535.71]

larger in magnitude than that in Example 5.9. The input, $\mathbf{u}(t) = -\mathbf{K}\mathbf{x}(t)$, can be calculated from the previously calculated matrices, $\mathbf{K}$ and $\mathbf{x}$, as follows:

```
>>u = -K*X'; <enter>
```

The control inputs for the two values of the regulator gain matrix are compared in Figure 5.6. The control input, $u(t)$, which is a force applied to the cart, is seen to be *more than 200 times* in magnitude for the design of Example 5.10 than that of Example 5.9. The *actuator*, which applies the input force to the cart, must be physically able to generate this force for the design to be successful. The cost of controlling a plant is a function of the largest control input magnitude expected in *actual operating conditions*. For example, if the largest *expected* initial disturbance in cart displacement were 0.1 m instead of 0.01 m, a *ten times larger* control input would be required than that in Figure 5.6. The larger the control input magnitude, the bigger would be the energy spent by the actuator in generating the control input, and the higher would be the cost of control. It is possible to *minimize* the control effort required in controlling a plant by imposing conditions – other than pole-placement – on the regulator gain matrix, which we will see in Chapter 6. However, a rough method of ensuring that the performance requirements are met with the minimum control effort is to ensure that all the closed-loop poles are about the *same distance* from the imaginary axis in the left-half plane. The poles in the left-half plane that are *farthest* away from the imaginary axis dictate the control input magnitude, while the *speed* of response (i.e. the settling time of the transients) is governed by the poles with the *smallest* real parts, called the *dominant poles*. If some closed-loop poles are close to, and some are very far from the imaginary axis, it implies that *too much control energy* is being spent for a given settling time, and thus the design is *inefficient*. The most efficient closed-loop configuration thus appears to be the one where all the poles are placed in the

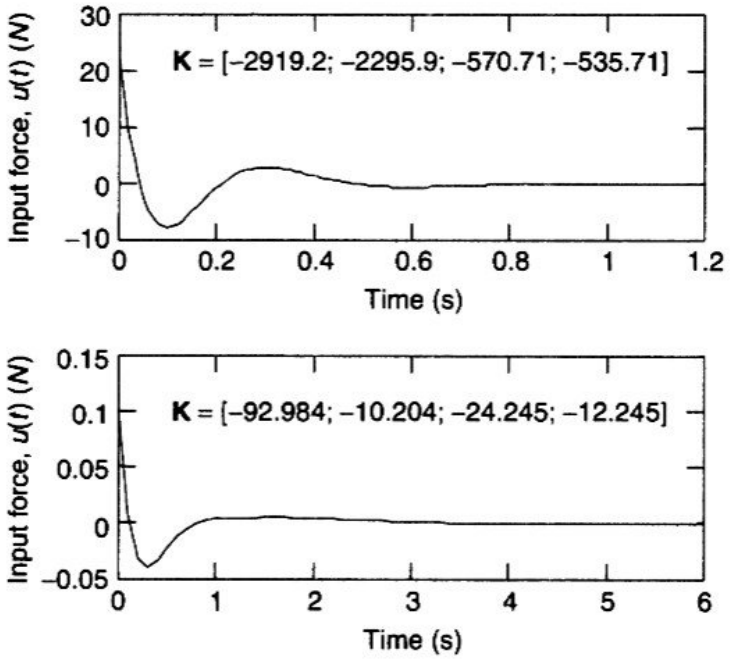


Figure 5.6 Control inputs of the regulated inverted pendulum on a moving cart for two designs of the full-state feedback regulator

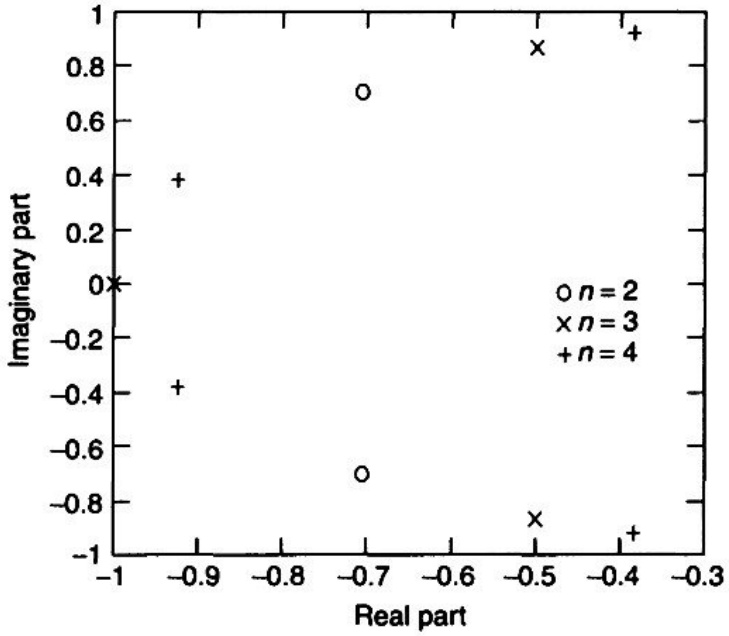


Figure 5.7 Butterworth pattern of poles in the left-half plane for $n = 2$, 3, and 4

left half plane, roughly the same distance from the imaginary axis. To increase the speed of the closed-loop response, one has to just increase this distance. One commonly used closed-loop pole configuration is the *Butterworth pattern*, in which the poles are placed on a circle of radius R *centered at the origin*, and are obtained from the solution of the following equation:

$$(s/R)^{2n} = (-1)^{n+1} \tag{5.59}$$

where n is the number of poles in the left-half plane (usually, we want all the closed-loop poles in the left-half plane; then n is the order of the system). For $n = 1$, the pole in the left-half plane satisfying Eq. (5.59) is $s = -R$. For $n = 2$, the poles in the left-half plane satisfying Eq. (5.59) are the solutions of $(s/R)^2 + (s/R)\sqrt{2} + 1 = 0$. The poles satisfying Eq. (5.59) in the left-half plane for $n = 3$ are the solutions of $(s/R)^3 + 2(s/R)^2 + 2(s/R) + 1 = 0$. For a given n, we can calculate the poles satisfying Eq. (5.59) by using the MATLAB function *roots*, and discard the poles having positive real parts. The Butterworth pattern for $n = 2$, 3, and 4 is shown in Figure 5.7. Note, however, that as n *increases*, the real part of the two Butterworth poles closest to the imaginary axis *decreases*. Thus for large n, it may be required to move these two poles further inside the left-half plane, in order to meet a given speed of response.

Example 5.11

Let us compare the closed-loop initial response and the input for the inverted pendulum on a moving cart with those obtained in Example 5.10 when the closed-loop poles are in a Butterworth pattern. For $n = 4$, the poles satisfying Eq. (5.59) in the left-half plane are calculated as follows: